



Facultad de Ingeniería y Ciencias Aplicadas

Escuela de Ingeniería de Software

Asignatura: Gestión De Configuración De Software
(ISW-115-1)

Herramientas De Control De Versiones

Sustentante: Rayber M. Reyes Donato (2023-3237)
Merali Santana De Jesus (2023-4415)
Richard A. Severino Morales (2019-0791)

Maestro: Francisco Loammi Mercedes Salvant

3 / 7 / 2026

Definición y Funcionalidades Principales

Para un correcto análisis arquitectónico, es fundamental separar los motores de control de versiones (el software local que rastrea los cambios) de las plataformas de alojamiento en la nube (que gestionan los repositorios remotos y la colaboración).

Git

Es un sistema de control de versiones distribuido (DVCS) de código abierto. Cada miembro del equipo posee una copia local completa del historial del proyecto, lo que elimina la dependencia de una red para realizar confirmaciones y optimiza drásticamente la velocidad.

- **Funcionalidades principales:**
 - Gestión avanzada y conmutación ultra rápida de ramas (branching y merging).
 - Arquitectura descentralizada con total independencia de conexión al servidor central para el trabajo diario.
 - Garantía estricta de integridad de los datos mediante el uso de hashes criptográficos.

GitHub

Es una plataforma de alojamiento en la nube optimizada para proyectos que utilizan el motor Git. Añade una capa de interfaz gráfica web, herramientas de gestión y características sociales orientadas al desarrollo.

- **Funcionalidades principales:**
 - Almacenamiento y centralización de repositorios remotos de Git.
 - Herramientas de revisión de código y flujo colaborativo mediante Pull Requests.
 - Gestión integrada de proyectos (Issues, tableros) y automatización de despliegues (GitHub Actions).

Subversion (SVN)

Un sistema de control de versiones de tipo centralizado (CVCS). Existe un único servidor principal que contiene todo el historial de cambios, y los desarrolladores bloquean o descargan únicamente los archivos específicos en los que están trabajando.

- **Funcionalidades principales:**
 - Control de acceso y permisos granular, configurable a nivel de carpetas o directorios específicos.
 - Historial unificado en un solo punto, ideal para auditorías corporativas rígidas.
 - Mecanismo de bloqueo de archivos para prevenir conflictos de edición simultánea.

Mercurial

Sistema de control de versiones distribuido diseñado en la misma época que Git, pero enfocado firmemente en la simplicidad, la consistencia de comandos y una curva de aprendizaje mucho más suave.

- **Funcionalidades principales:**
 - Interfaz de línea de comandos intuitiva, predecible y con menor riesgo de errores humanos de configuración.
 - Arquitectura modular extensible mediante complementos oficiales integrados.
 - Alta eficiencia en el manejo de proyectos masivos y archivos pesados.

Bazaar

Un sistema distribuido (DVCS) desarrollado originalmente por Canonical. Su propuesta de valor principal es la flexibilidad extrema, permitiendo adaptarse a flujos de trabajo muy diversos.

- **Funcionalidades principales:**
 - Soporte híbrido: Puede configurarse para operar de forma centralizada (estilo SVN) o distribuida (estilo Git).
 - Excelente integración nativa con entornos Linux y la plataforma Launchpad.
 - Fácil seguimiento de renombrado de archivos y directorios complejos.

Elección y Preferencia del Equipo

Nuestra herramienta de preferencia sería una combinación de Git (como motor local) + GitHub (como plataforma remota de colaboración).

Justificación:

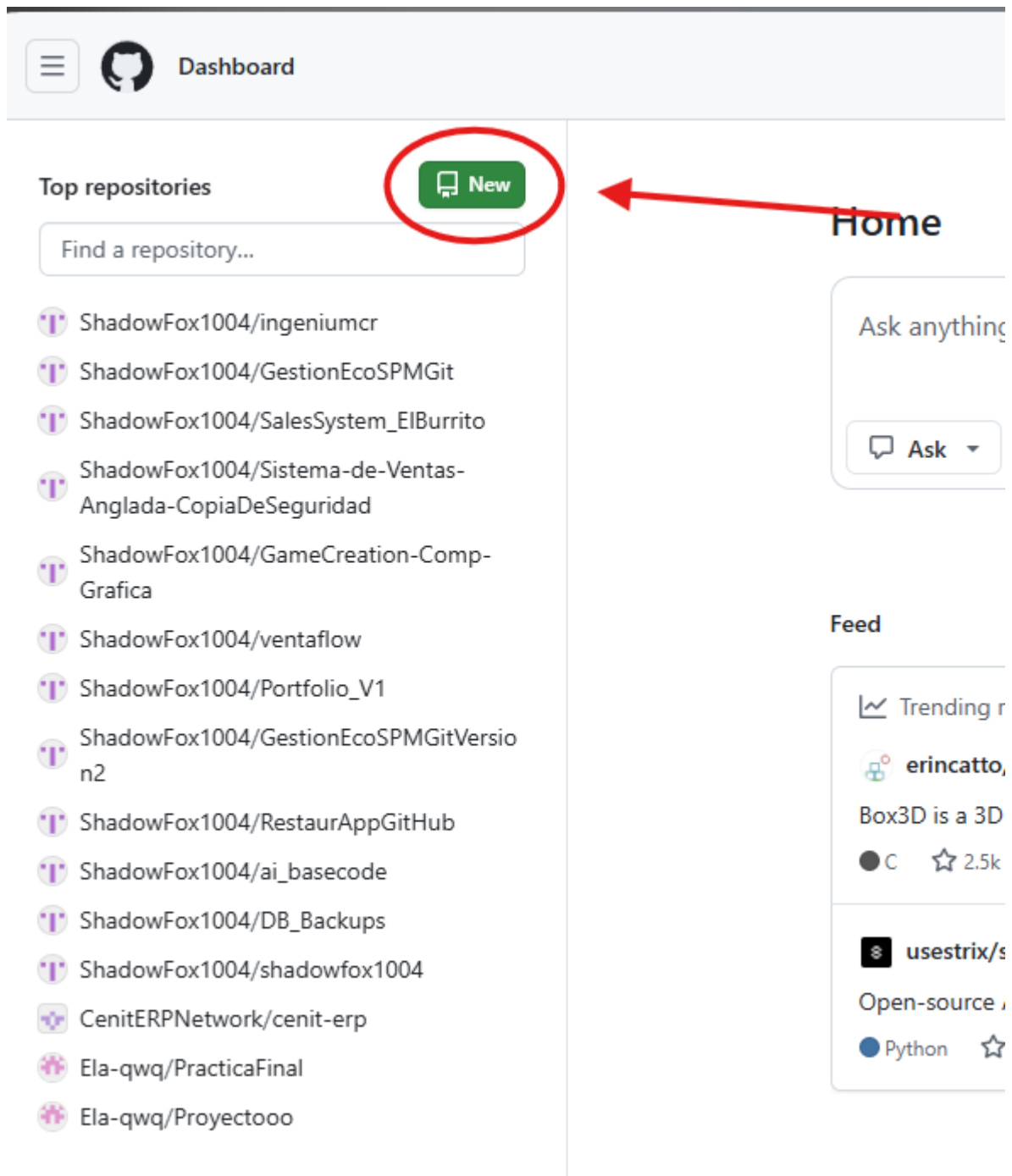
Al analizar las necesidades en el desarrollo de software, esta combinación destaca como el estándar indiscutible, ya que la arquitectura distribuida de Git nos permite experimentar de forma segura mediante ramas independientes para implementar lógicas de negocio complejas (como estructuras multicapa o componentes de persistencia) sin alterar el código estable del sistema principal.

Por otro lado, tal como se debatió en la sesión de clase, GitHub eleva el trabajo colaborativo al permitir el establecimiento de roles claros, asignación de privilegios de seguridad y restricciones de fusión. Su ecosistema facilita la revisión mutua de código antes de integrar cambios al proyecto base, minimizando errores y asegurando la contabilidad del estado del software.

Definición de los Principales Comandos (Basado en Git/GitHub)

- `git init`: Inicializa un nuevo repositorio Git local en la carpeta del proyecto actual.
- `git clone <URL>`: Descarga una copia exacta e idéntica de un repositorio alojado en GitHub hacia la máquina local.
- `git status`: Muestra el estado del directorio de trabajo, identificando qué archivos han sido modificados, cuáles están listos para confirmarse (Staging Area) y cuáles no están bajo seguimiento.
- `git add <archivo>`: Traslada los cambios de un archivo específico al área de preparación (Staging Area). El uso de `git add .` indexa todas las modificaciones de la carpeta.
- `git commit -m "Mensaje descriptivo"`: Consolida los cambios del Staging Area guardándolos permanentemente en el historial del repositorio local junto con un comentario explicativo.
- `git branch`: Permite listar, crear o eliminar las ramificaciones del proyecto.
- `git checkout <nombre_de_rama>`: Modifica el entorno de trabajo apuntando a la rama seleccionada para visualizar o editar su código específico.
- `git merge <nombre_de_rama>`: Fusiona e integra el historial de la rama especificada dentro de la rama activa en la que se encuentra el desarrollador.
- `git push origin <nombre_de_rama>`: Sube y transfiere los commits registrados localmente hacia el repositorio remoto guardado en GitHub.
- `git pull`: Descarga las últimas actualizaciones del repositorio remoto en GitHub y las fusiona de forma inmediata con la copia local.
- `git stash`: Almacena temporalmente las modificaciones locales no confirmadas en una pila provisional, limpiando el directorio para permitir cambios de rama de emergencia sin perder el progreso.
- `git stash pop`: Recupera y aplica nuevamente en el espacio de trabajo los últimos cambios guardados provisionalmente mediante el comando `stash`.

Ejecución de Procedimientos



The screenshot shows the GitHub Dashboard interface. At the top, there is a header bar with a menu icon, the GitHub logo, and the word "Dashboard". Below the header, the main content area is divided into two columns. The left column is titled "Top repositories" and contains a search bar labeled "Find a repository...". Below the search bar is a list of repositories, each with a repository icon and a name. The right column is titled "Home" and contains a section labeled "Ask anything" with a search bar and a button labeled "Ask". Below this is a section labeled "Feed" which displays trending repositories. A red circle highlights the "New" button in the top right corner of the "Top repositories" section. A red arrow points from the word "Home" to the "New" button.

Dashboard

Top repositories

New

Find a repository...

ShadowFox1004/ingeniumcr

ShadowFox1004/GestionEcoSPMGit

ShadowFox1004/SalesSystem_ElBurrito

ShadowFox1004/Sistema-de-Ventas-Anglada-CopiaDeSeguridad

ShadowFox1004/GameCreation-Comp-Grafica

ShadowFox1004/ventaflow

ShadowFox1004/Portfolio_V1

ShadowFox1004/GestionEcoSPMGitVersion2

ShadowFox1004/RestaurAppGitHub

ShadowFox1004/ai_basecode

ShadowFox1004/DB_Backups

ShadowFox1004/shadowfox1004

CenitERPNetwork/cenit-erp

Ela-qwq/PracticaFinal

Ela-qwq/Proyecto000

Home

Ask anything

Ask

Feed

Trending repositories

erincatto

Box3D is a 3D

C 2.5k

usestrix/s

Open-source

Python

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner *

 ShadowFox1004 ▾

Repository name *

Control_De_Versiones_ISW_115_1

✓ Control_De_Versiones_ISW_115_1 is available.

Great repository names are short and memorable. How about [verbose-octo-robot](#)?

Description

Tarea3 Herramientas de control de versiones

43 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

On ☒

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore ▾

Add license

Licenses explain how others can use your code. [About licenses](#)

No license ▾

Create repository

Control_De_Versiones_ISW_115_1 Public

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file

Add file <> Code

ShadowFox1004 Initial commit ba6f79d · now 1 Commit

README.md Initial commit now

README

Control_De_Versiones_ISW_115_1

Tarea3 Herramientas de control de versiones

About

Tarea3 Herramientas de control de versiones

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 1

ShadowFox1004

Control_De_Versiones_ISW_115_1 Public

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file

Add file <> Code

ShadowFox1004 Initial commit

README.md Initial commit

README

Control_De_Versiones_ISW_115_1

Tarea3 Herramientas de control de versiones

About

Tarea3 Herramientas de control de versiones

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 1

ShadowFox1004

Local Codespaces

Clone

HTTPS SSH GitHub CLI

https://github.com/ShadowFox1004/Control_De_Ver

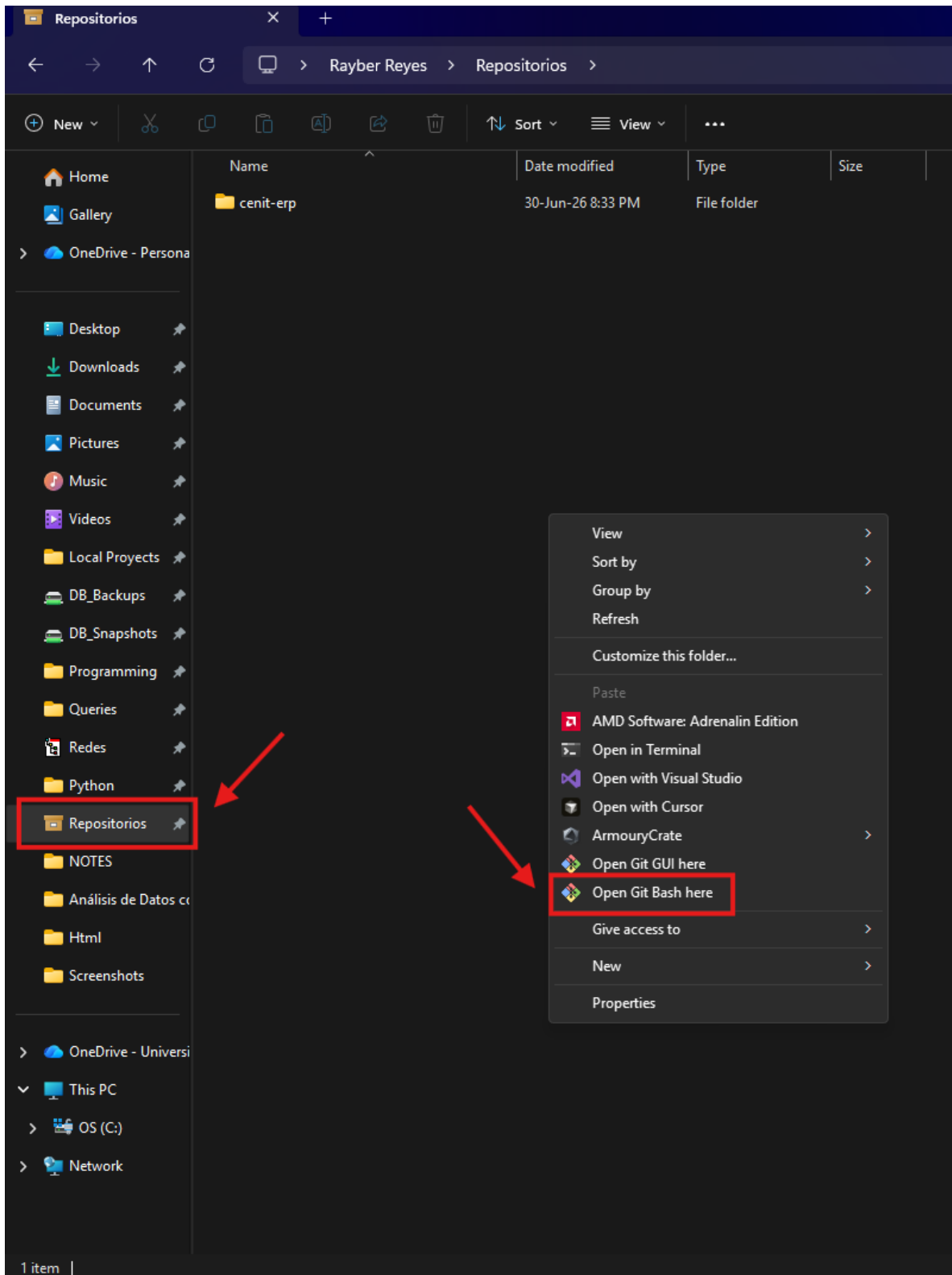
Clone using the web URL

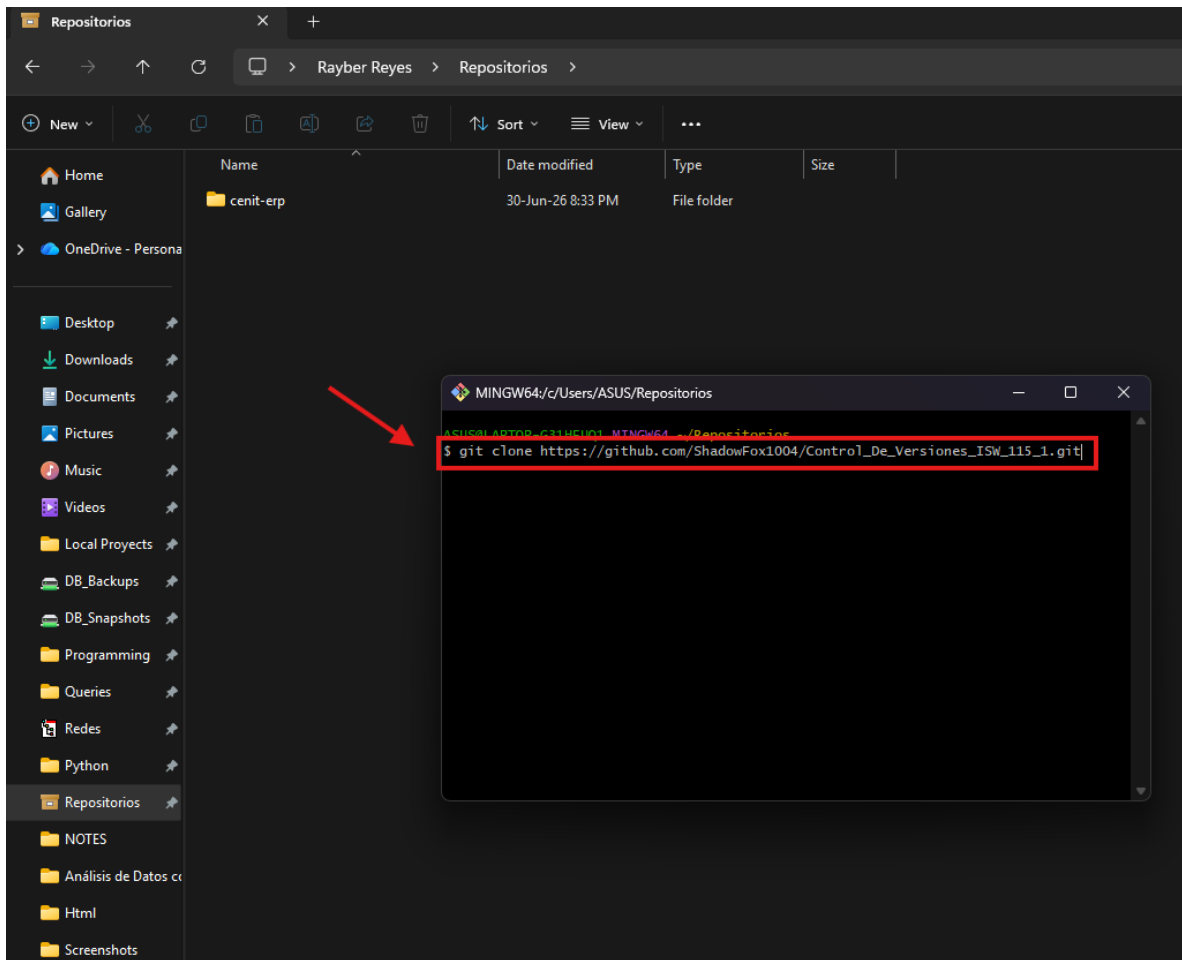
Open in GitHub Copilot app

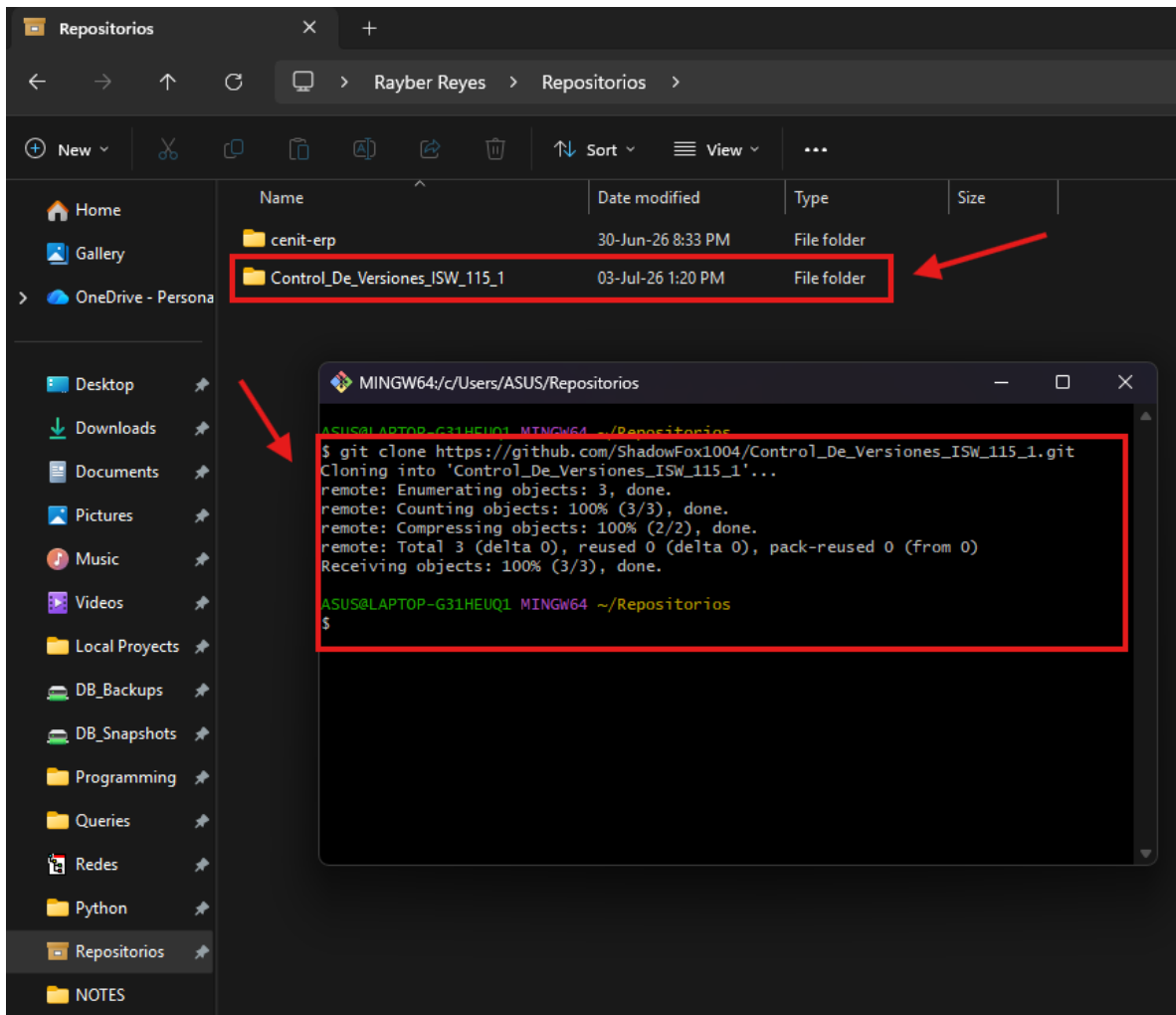
Open with GitHub Desktop

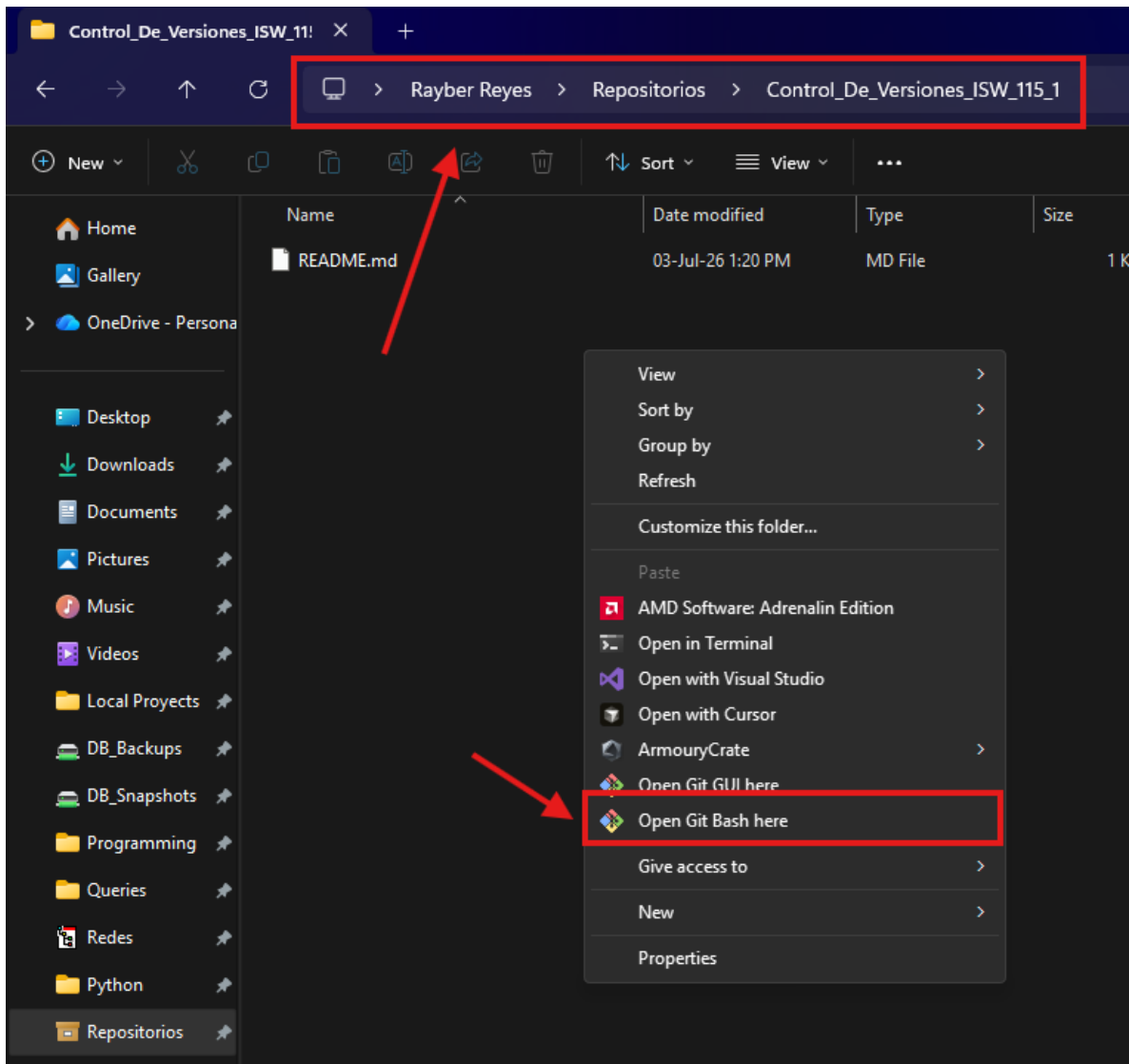
Open with Visual Studio

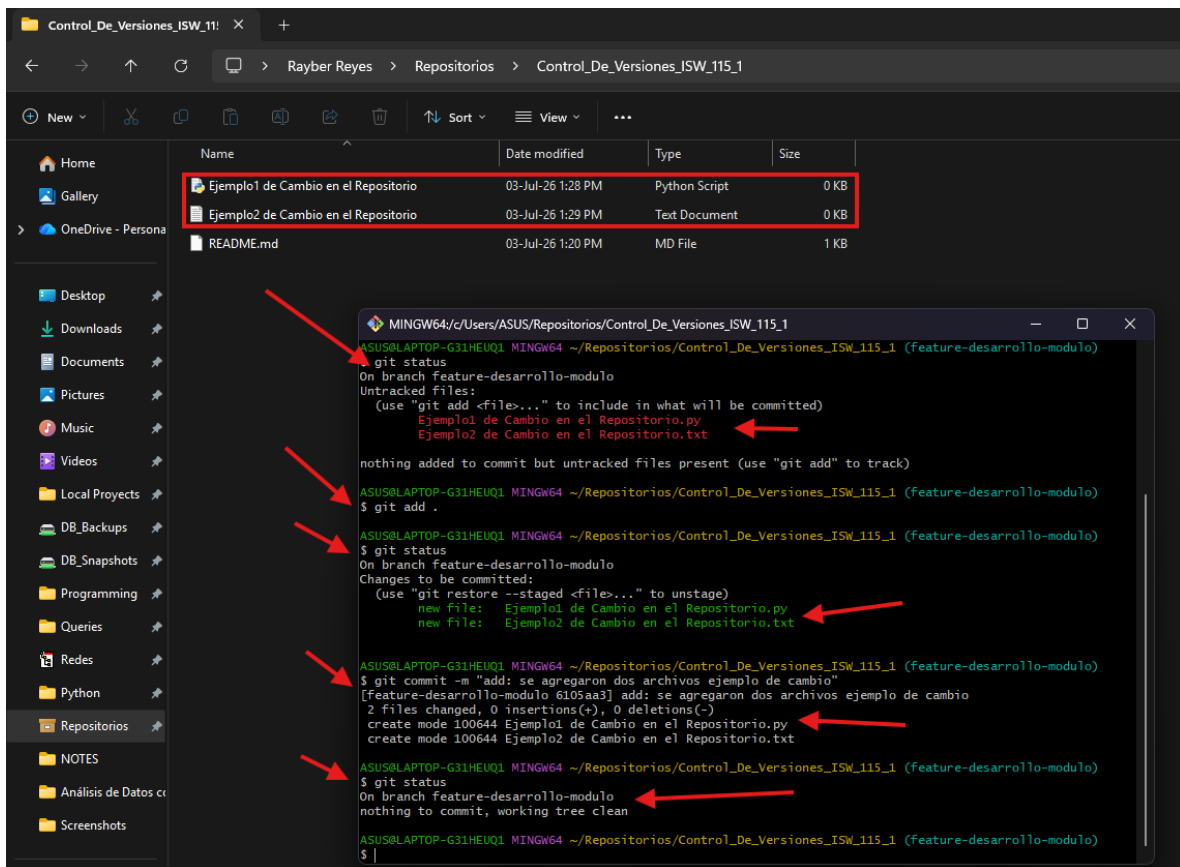
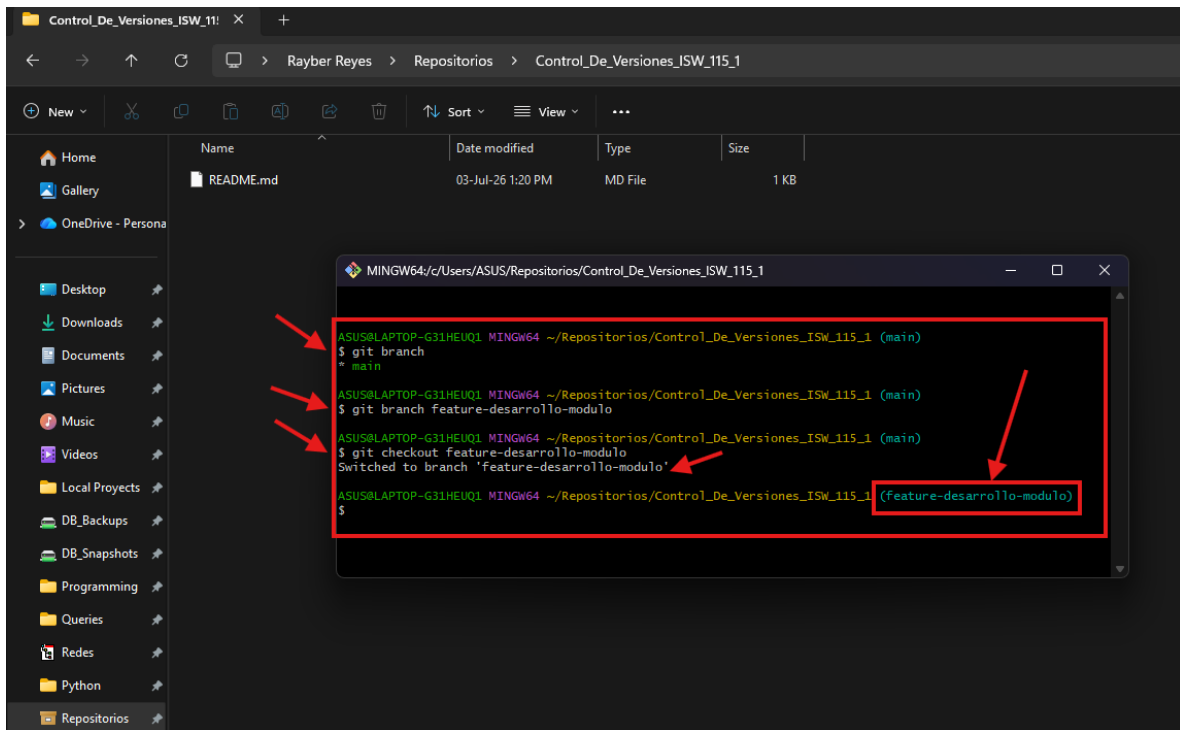
Download ZIP

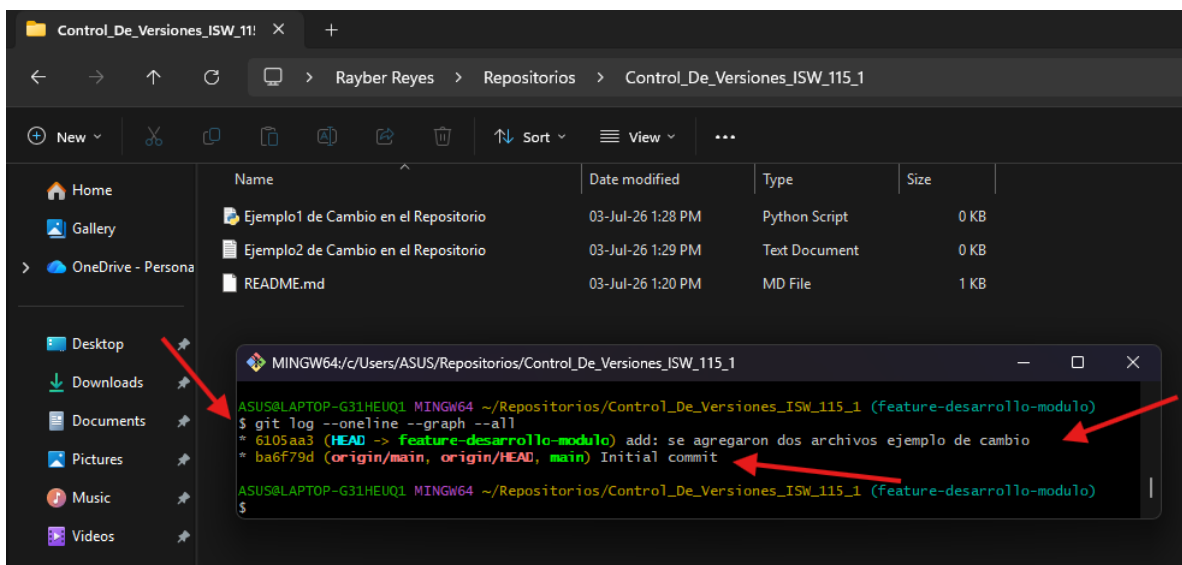
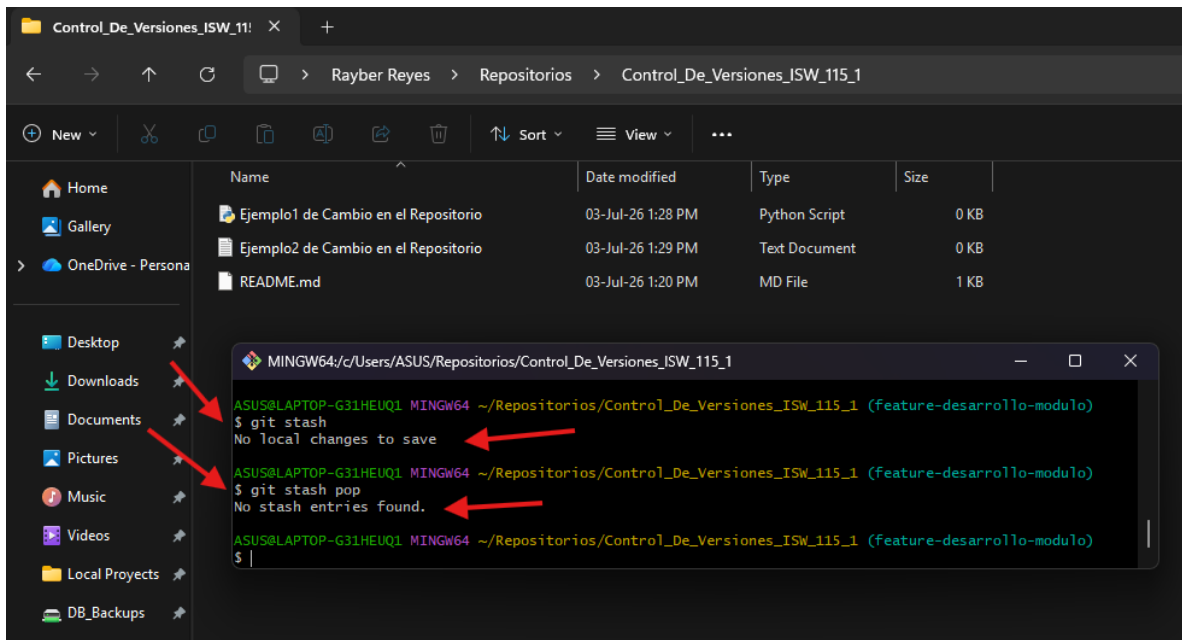


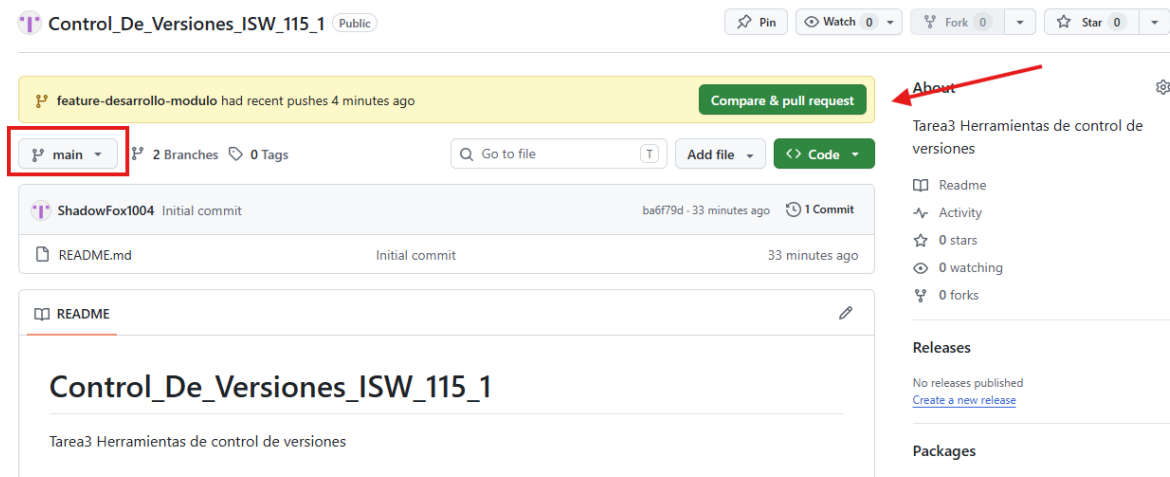
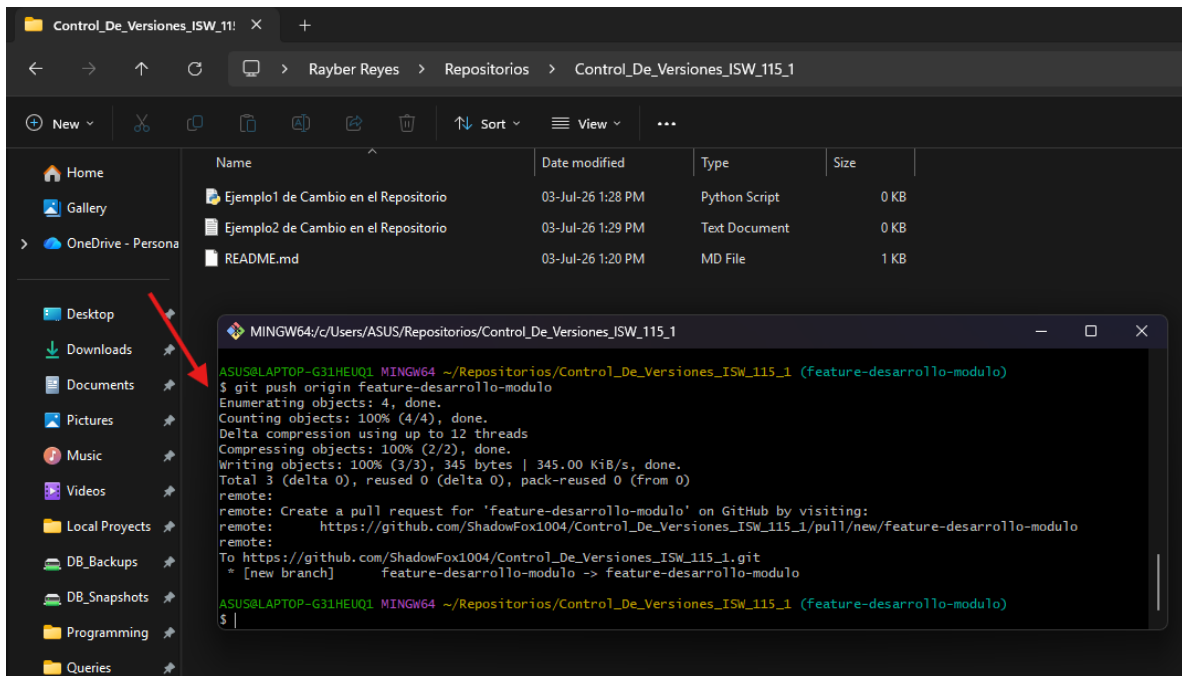












feature-desarrollo-modulo had recent pushes 2 minutes ago [Compare & pull request](#)

feature-desarrollo-modulo 2 Branches 0 Tags Add file Code

This branch is 1 commit ahead of main. Contribute

ShadowFox1004	add: se agregaron dos archivos ejemplo de cambio	6105aa3 · 11 minutes ago	2 Commits
Ejemplo1 de Cambio en el Repositorio.py	add: se agregaron dos archivos ejemplo de cambio	11 minutes ago	
Ejemplo2 de Cambio en el Repositorio.txt	add: se agregaron dos archivos ejemplo de cambio	11 minutes ago	
README.md	Initial commit	31 minutes ago	

README

Control_De_Versiones_ISW_115_1

Tarea3 Herramientas de control de versiones

About

Tarea3 Herramientas de control de versiones

- Readme
- Activity
- 0 stars
- 0 watching
- 0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors 1

ShadowFox1004

Todos los comandos que se ejecutaron:

```
MINGW64/c/Users/ASUS/Repositorios/Control_De_Versiones_ISW_115_1

ASUS@LAPTOP-G31HEUQ1 MINGW64 ~/Repositorios/Control_De_Versiones_ISW_115_1 (main)
$ git branch
* main

ASUS@LAPTOP-G31HEUQ1 MINGW64 ~/Repositorios/Control_De_Versiones_ISW_115_1 (main)
$ git branch feature-desarrollo-modulo

ASUS@LAPTOP-G31HEUQ1 MINGW64 ~/Repositorios/Control_De_Versiones_ISW_115_1 (main)
$ git checkout feature-desarrollo-modulo
Switched to branch 'feature-desarrollo-modulo'

ASUS@LAPTOP-G31HEUQ1 MINGW64 ~/Repositorios/Control_De_Versiones_ISW_115_1 (feature-desarrollo-modulo)
$ git status
On branch feature-desarrollo-modulo
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    Ejemplo1 de Cambio en el Repositorio.py
    Ejemplo2 de Cambio en el Repositorio.txt

nothing added to commit but untracked files present (use "git add" to track)

ASUS@LAPTOP-G31HEUQ1 MINGW64 ~/Repositorios/Control_De_Versiones_ISW_115_1 (feature-desarrollo-modulo)
$ git add .

ASUS@LAPTOP-G31HEUQ1 MINGW64 ~/Repositorios/Control_De_Versiones_ISW_115_1 (feature-desarrollo-modulo)
$ git status
On branch feature-desarrollo-modulo
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   Ejemplo1 de Cambio en el Repositorio.py
    new file:   Ejemplo2 de Cambio en el Repositorio.txt

ASUS@LAPTOP-G31HEUQ1 MINGW64 ~/Repositorios/Control_De_Versiones_ISW_115_1 (feature-desarrollo-modulo)
$ git commit -m "add: se agregaron dos archivos ejemplo de cambio"
[feature-desarrollo-modulo 6105aa3] add: se agregaron dos archivos ejemplo de cambio
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Ejemplo1 de Cambio en el Repositorio.py
create mode 100644 Ejemplo2 de Cambio en el Repositorio.txt

ASUS@LAPTOP-G31HEUQ1 MINGW64 ~/Repositorios/Control_De_Versiones_ISW_115_1 (feature-desarrollo-modulo)
$ git status
On branch feature-desarrollo-modulo
nothing to commit, working tree clean

ASUS@LAPTOP-G31HEUQ1 MINGW64 ~/Repositorios/Control_De_Versiones_ISW_115_1 (feature-desarrollo-modulo)
$ git stash
No local changes to save

ASUS@LAPTOP-G31HEUQ1 MINGW64 ~/Repositorios/Control_De_Versiones_ISW_115_1 (feature-desarrollo-modulo)
$ git stash pop
No stash entries found.

ASUS@LAPTOP-G31HEUQ1 MINGW64 ~/Repositorios/Control_De_Versiones_ISW_115_1 (feature-desarrollo-modulo)
$ git log --oneline --graph --all
* 6105aa3 (HEAD -> feature-desarrollo-modulo) add: se agregaron dos archivos ejemplo de cambio
* ba6f79d (origin/main, origin/HEAD, main) Initial commit

ASUS@LAPTOP-G31HEUQ1 MINGW64 ~/Repositorios/Control_De_Versiones_ISW_115_1 (feature-desarrollo-modulo)
$ git push origin feature-desarrollo-modulo
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 345 bytes | 345.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-desarrollo-modulo' on GitHub by visiting:
remote:   https://github.com/ShadowFox1004/Control_De_Versiones_ISW_115_1/pull/new/feature-desarrollo-modulo
remote:
To https://github.com/ShadowFox1004/Control_De_Versiones_ISW_115_1.git
 * [new branch]   feature-desarrollo-modulo -> feature-desarrollo-modulo
```